

Deep Learning 3

Architectures and Applications

Machine learning

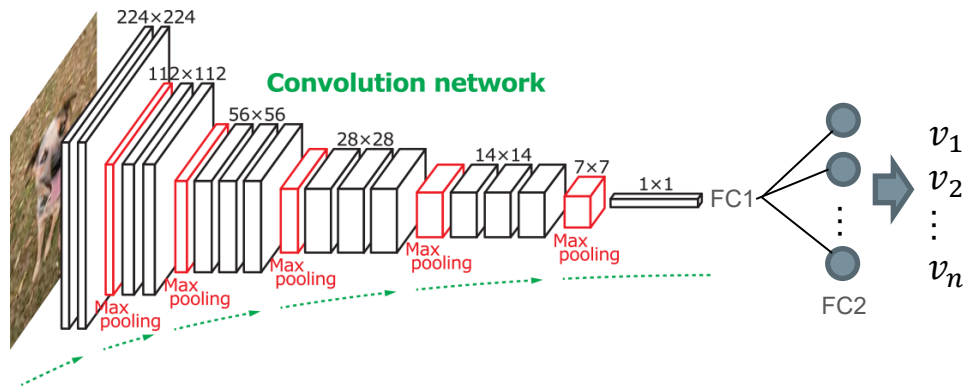
Jiří Chmelík & Roman Jakubíček
Department of Biomedical Engineering
Brno University of Technology

Regression and classification

Architecture

Regression problem

- Architecture: encoder => FC
- Input: image
- Output: regressed values
- Loss fcn: L_1 or L_2
- Last layer:
 - fully connected / 1x1 conv
 - no activation function (in the specific cases ReLu or softmax)
 - number of neurons corresponds to the number of regressed values
- it is sometimes appropriate that the predicted value is relative, e.g. for prediction positions,...



Classification problem

- Architecture: encoder => FC
- Input: image
- Output: vector of probabilities

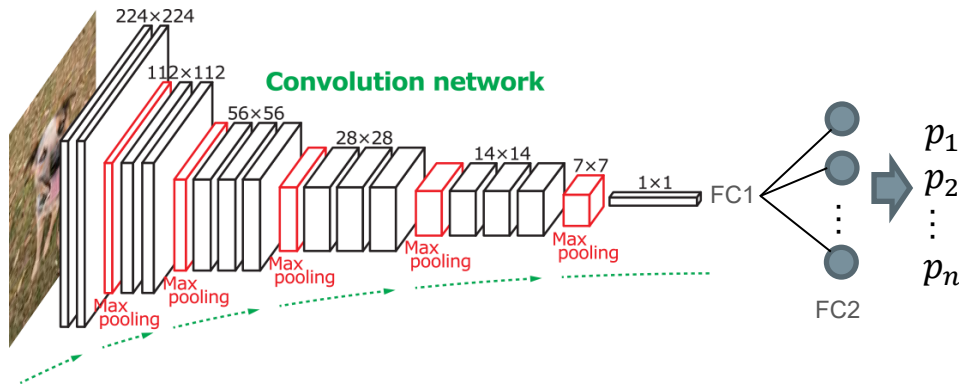
- Loss fcn: Multi-class Weighted Cross-entropy
- Last layer:
 - Fully connected / 1x1 conv
 - Softmax (sigmoid)
 - Number of neurons corresponds to the number of classes

One-hot encoding

- Output vector includes values of membership probabilities for each class (length of the vector equals the number of classes)
- Resulting estimated classes are taken with maximal probabilities

Integer encoding

- Alternatively, the output can be integer number – a label of class (e.g. suitable for ordinal variable)



Encoders

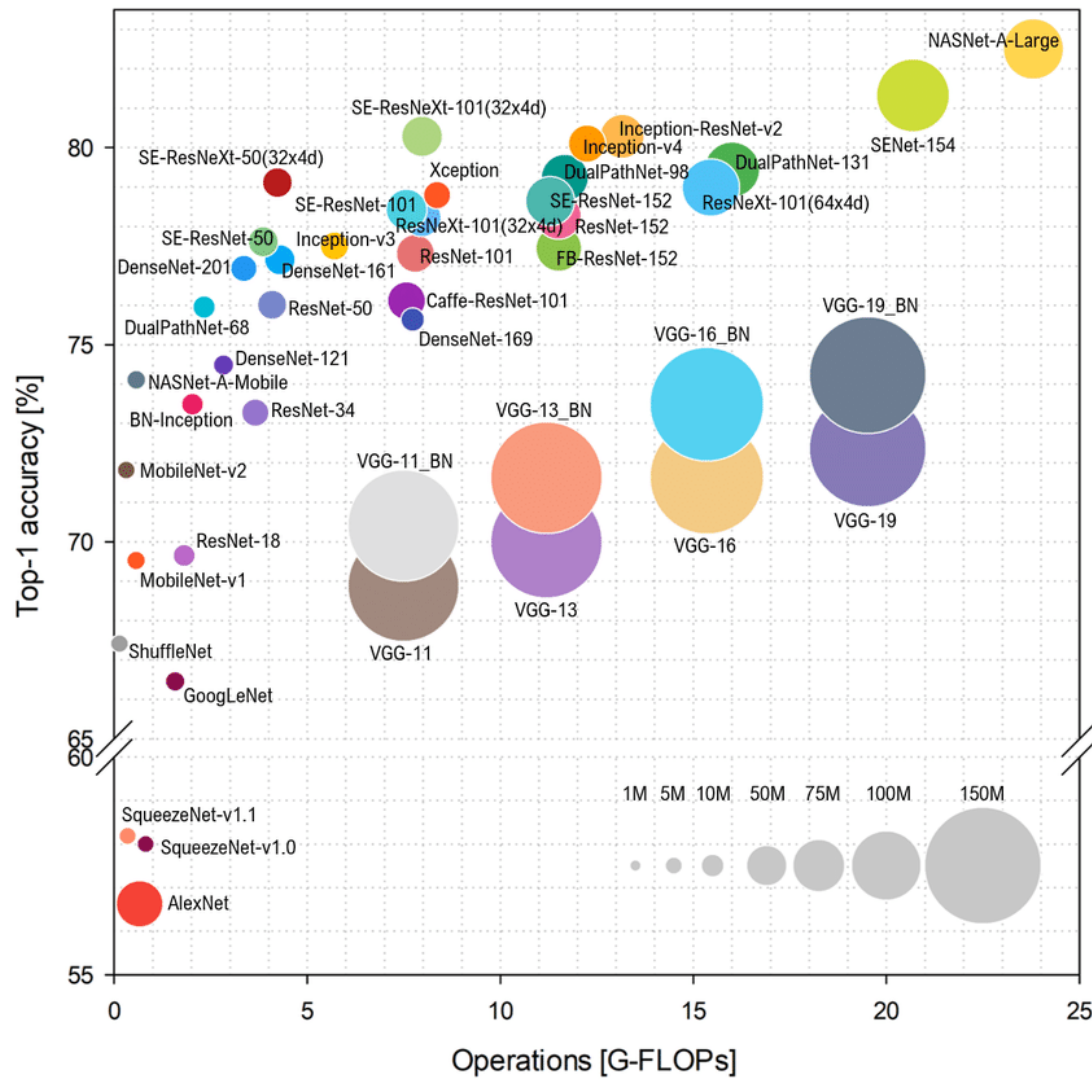
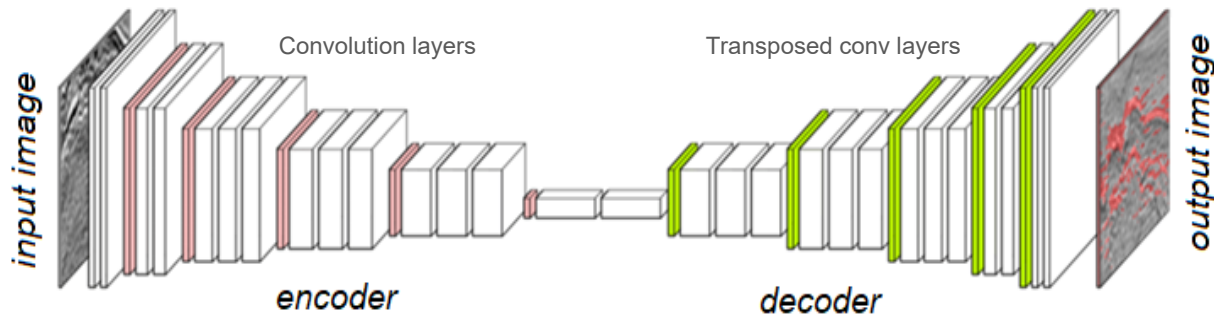


Image-to-image

Architecture

Image Regression

- Architecture: encoder – decoder
- Input: image
- Output: image
- Loss fcn: L_1 or L_2
- Last layer:
 - Convolution layer
 - Linear activation



- We need paired data (reference data for an input image) – can be problematic
- If we haven't these data, we can create e.g. synthetic or obtain during new acquisition
- There are existing special methods, which work with non-paired images

Image Regression

Utilization:

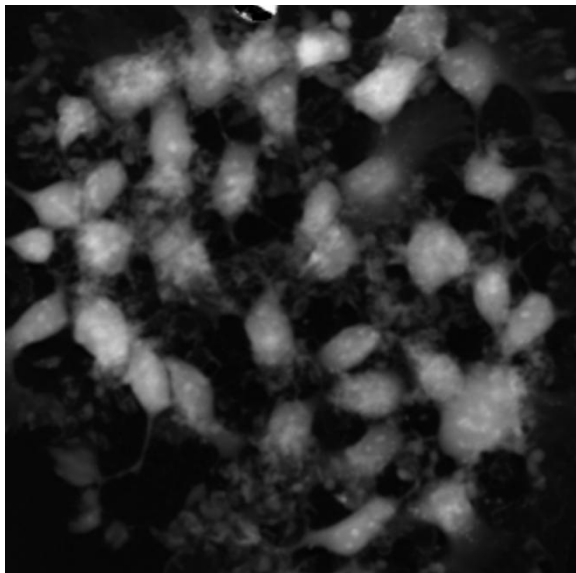
- image segmentation
- noise reduction
- reconstruction from sinogram
- image transformation to new image (GAN,...)



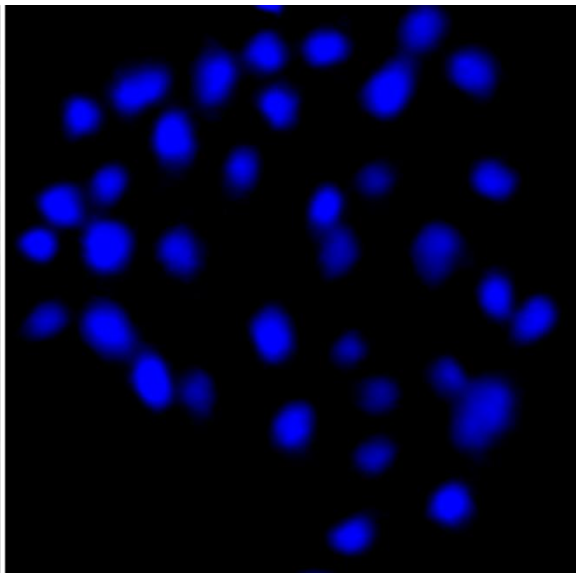
Image Regression

Example of fluorescence prediction of cells' nuclei from microscopic image

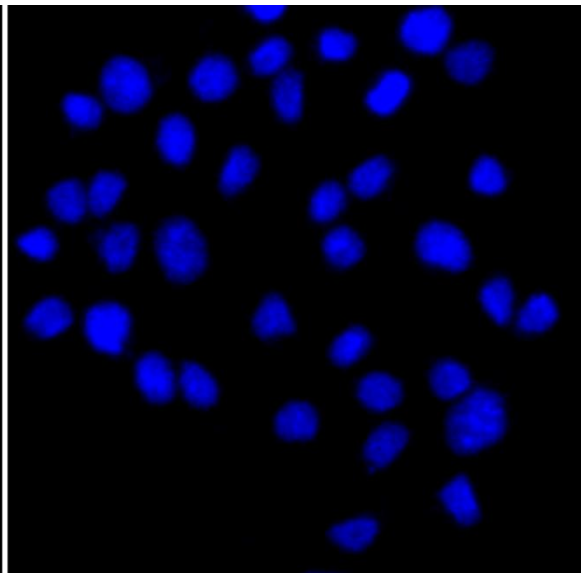
Input image



Regression image



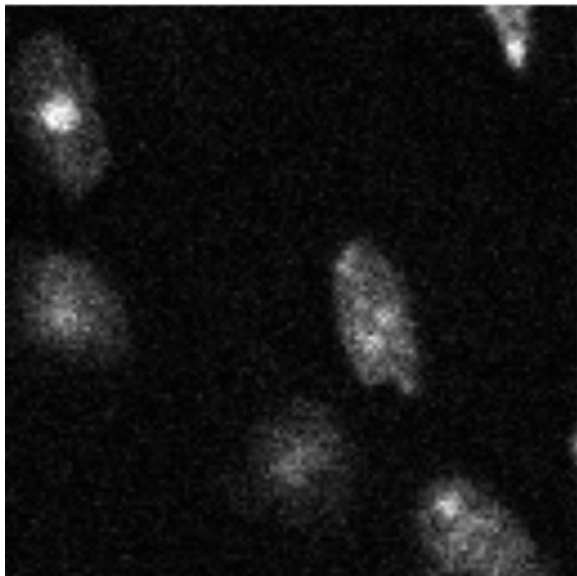
Reference image



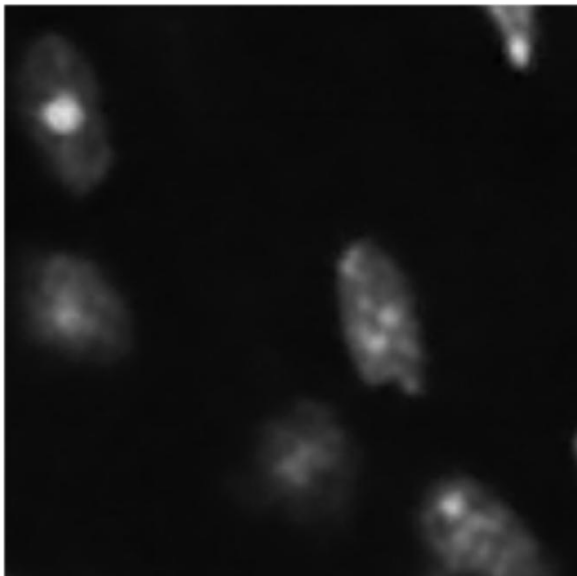
Noise reduction

Example of denoising

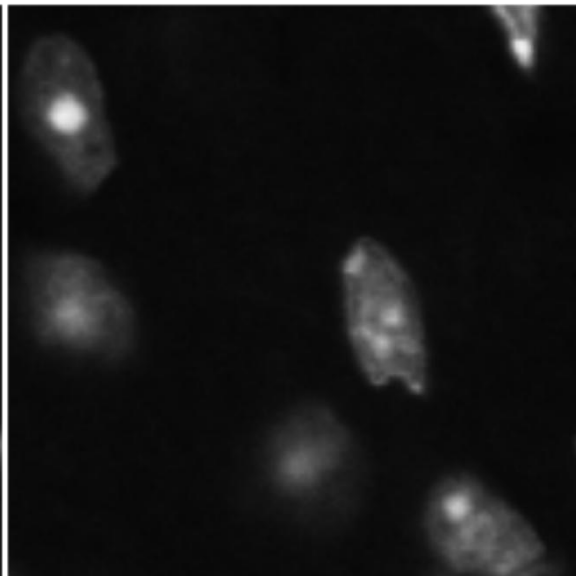
Input image



Regression image



Reference image



Next example:

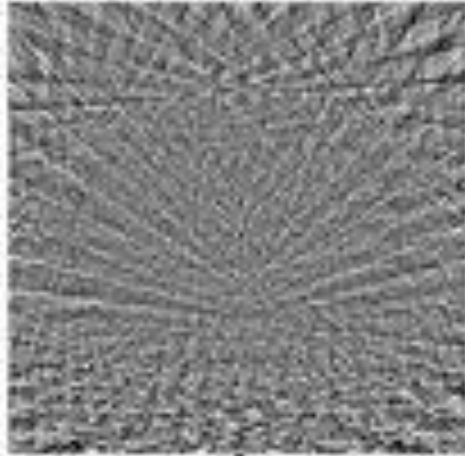
https://www.youtube.com/watch?time_continue=24&v=ooeHI_qmqzl&feature=emb_logo

Noise reduction

Input image



Regression image



Resulting image



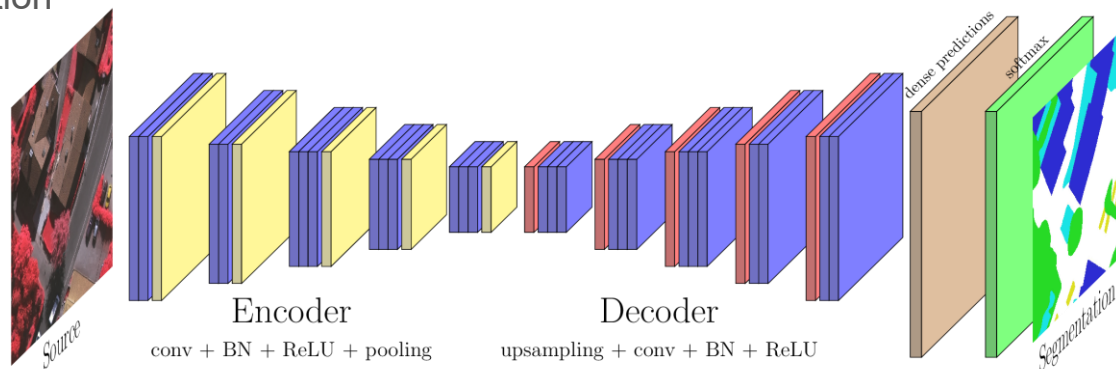
Image segmentation

A special case of the image-to-image transformation – output is a label of the classes for each pixel

- **Architecture:** encoder – decoder
- **Input:** image
- **Output:** 2D probability map for each class
- **Loss fcn:** Multi-class Weighted Cross-entropy or Binary Tverski (e.g. Dice loss)
- **Last layer:**
 - convolution layer, where its depth size = number of classes
 - softmax (through depth) for each pixel
 - alternatively 1x1 convolution

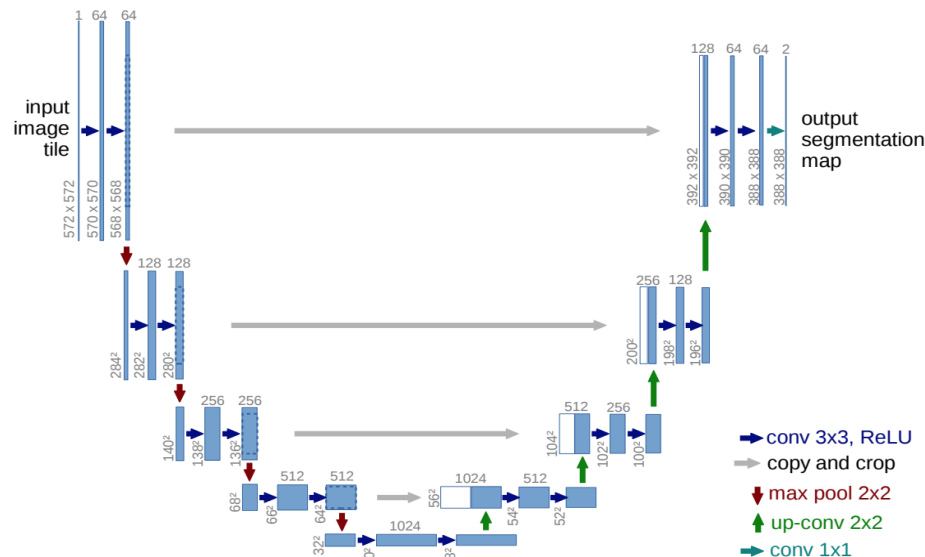
Output has size $M \times N \times D$, where D is the number of classes.

We select classes with maximal probability for each pixel.

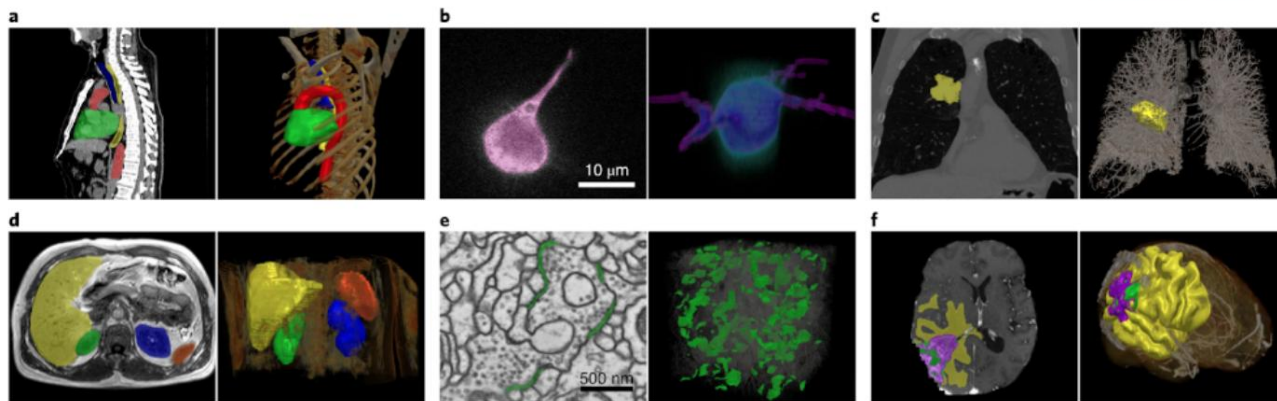
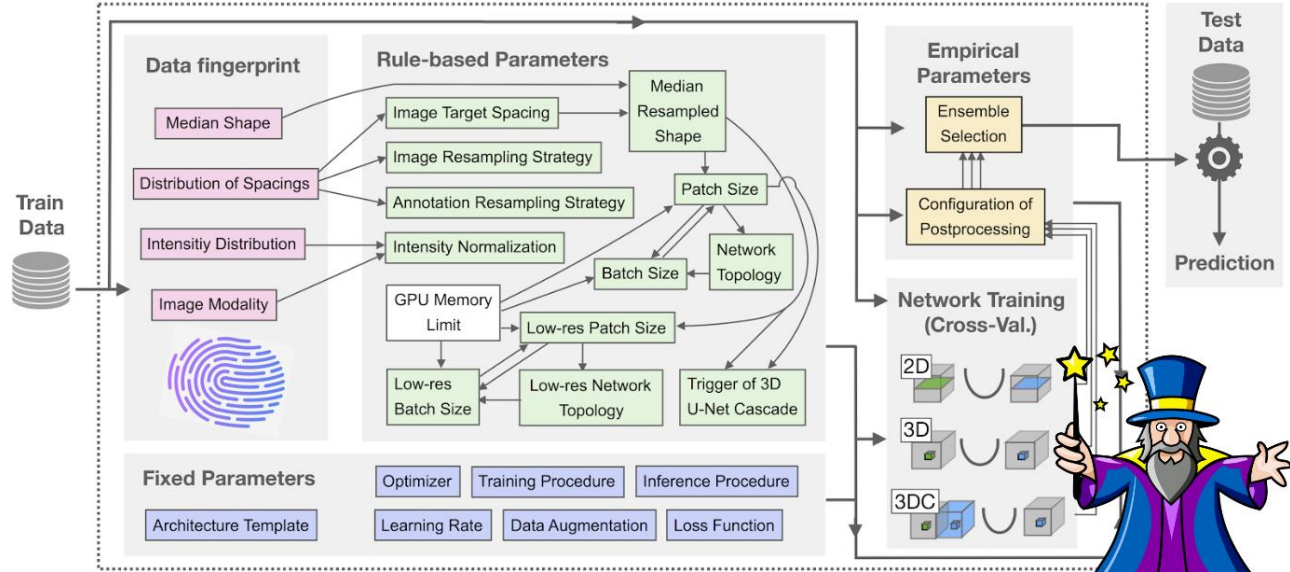


U-Net (2015)

- Developed for biomedical image segmentation
- Use long **skip connections** between the down-sampling path and the up-sampling path
- Skip connections with **concatenation**
- The bottleneck is built from two 3x1 convolutional layers (with BN and ReLu)
- It combines the location information from the down-sampling path with the contextual information in the up-sampling path
- No general pre-trained model
- There are many pre-trained models for specific segmentation problems (e.g. MRI brain)
- But, we can use e.g. VGG pre-trained model in the encoder part

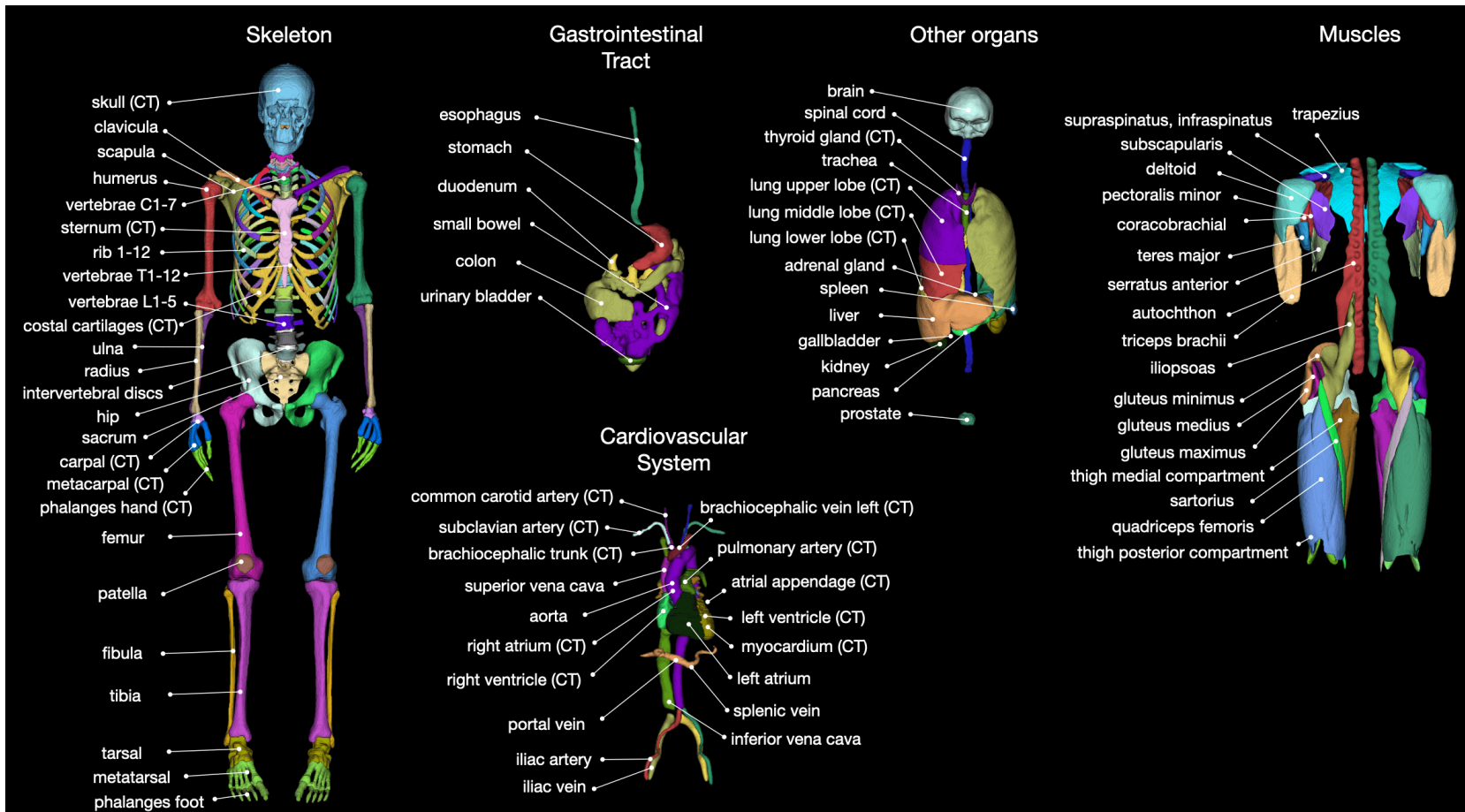


nnU-Net



TotalSegmentator

<https://pubs.rsna.org/doi/10.1148/ryai.230024>



Object detection

Architectures

Object detection

It is a computer vision technique that aims to detect objects in from bounding boxes. It can find objects such as cars, human, face or anatomical structure or pathologies.

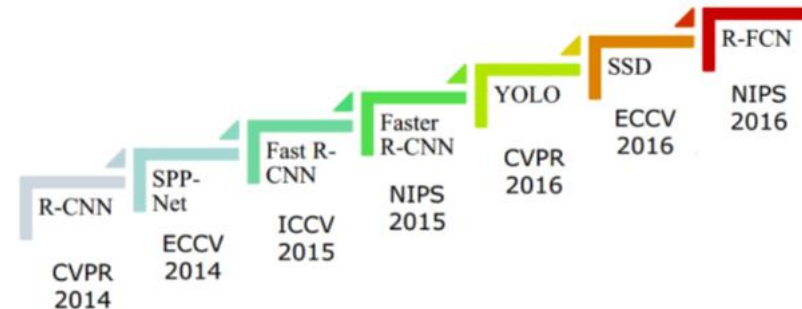
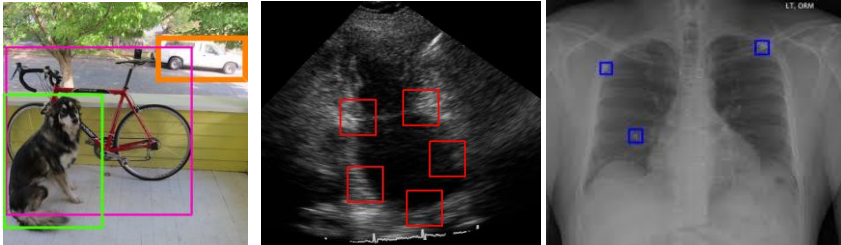
Object detection usually involves two processes:

- localization of the object
- classification of this object

Usually, these steps can be performed iteratively, parallel or in one step.

The most common algorithms:

- R-CNN (“fast”, “faster”, “cascade”)
- SSD
- YOLO (v.3) – v12 newest
- RetinaNet
- and much more...



Faster R-CNN

Region convolution neural network

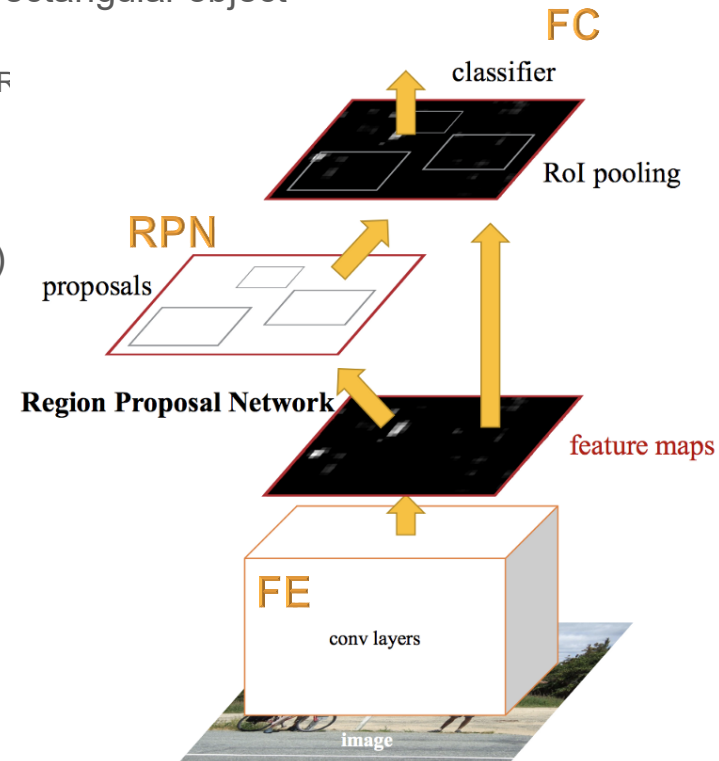
- Takes an image (of any size) as input and outputs a set of rectangular object proposals, each with an objectness score
- The third version of the algorithm (R-CNN => fast R-CNN => faster F

Basic two modules:

- extraction of features by CNN with FC classifier (fRCNN)
- network for proposal region (RPN)

- The iterative four-stage learning process
- Sharing convolution layers

Modules: FE – features extractor, FC – fully connected classificatory, RPN – region proposal network

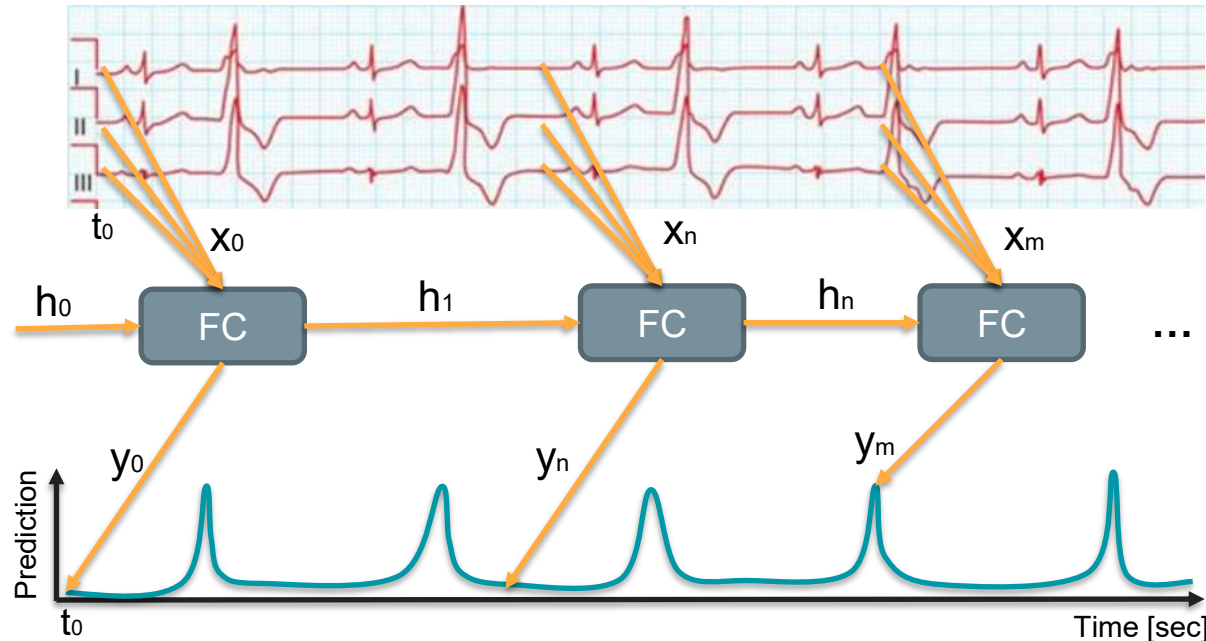


Signal and Language

Architectures

Recurrent Networks

- this network introduces new input ($\mathbf{h}$) vector, which is output from network in previous time point
- FC has thus two input vectors ($\mathbf{h}_{k-1}$ and $\mathbf{x}_{k-1}$) and two outputs ($\mathbf{h}_k$ and $\mathbf{y}_k$)
- for prediction of next time point in signals through any dimension (time), the FC net uses information in form the vector $\mathbf{h}$ from previous time point



Summary

Pros

- predictor, which uses older prediction in the form activation values (h_{i-1}) from previous “iteration”
- learned gradual forgetting older prediction
- information transfer over time (or another dimension)
- we can process sequence of different lengths
- if the input size is larger, the model size does not increase

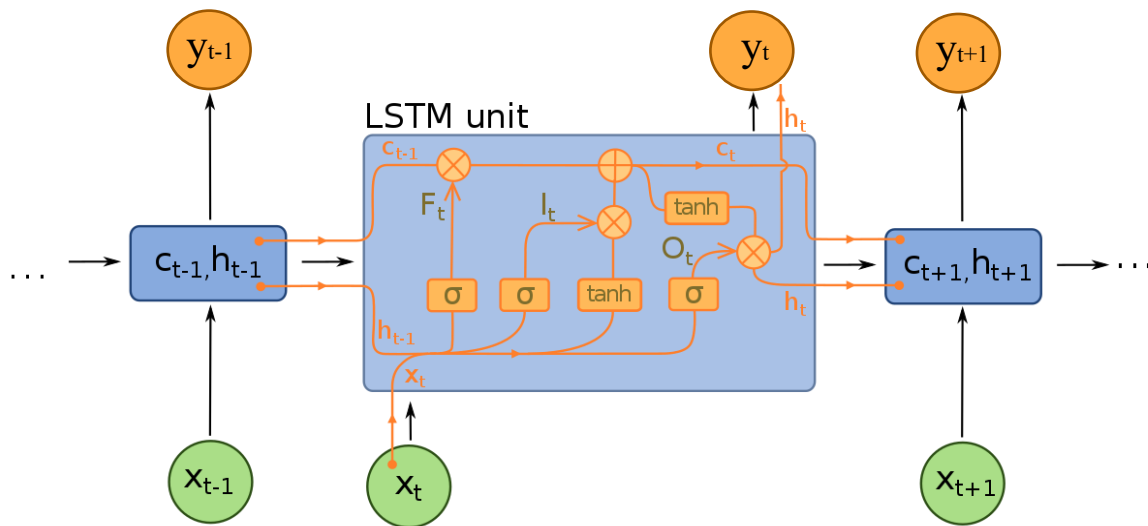
Cons

- difficult training (depend on length of signals)
- for each “time” point, the prediction of networks is required – must be performed iteratively
- gradient vanishing and exploding problems
- cannot be parallelized
- cannot be used transfer learning

There are many modifications of RNNs (e.g., network doesn't use $\mathbf{W}_y$ or bi-directional, ...)

LSTM – Long Short-Term Memory

- This network introduces into RNN the recurrent “forget gates”, which allow use very long memories.
- The LSTM approach solves vanishing gradient or exploding problem in backprop.
- It can work even given long delay between significant events



Summary

Pros

- very successful modification of RNNs and their utilizing
- solves a vanishing gradient problem (error can be backpropagated through time and layers)
- long memory
- we can process sequence of different lengths

Cons

- uses sigmoid or tanh activation function – more prone to saturation (you can use “softsign”)
- Iterative calculation – high computational complexity
- cannot be parallelized
- cannot use transfer learning
- cannot be re-trained – generalization problem

There are many modifications of the LSTM (e.g. adding “peephole connection”, Depth Gated RNNs, Clockwork RNNs, grid LSTM or **GRU**)

Utilizing

- **Natural language processing**
 - obtaining the context from sentence/text
 - classification of text document
 - automatic translation
 - spam detection
- **Speech recognition**
 - transfer spoken word to written text
- **Financial prediction, ...**
- **Analysis of sequences:**
 - gene sequences (e.g. secondary structure prediction, ...)
 - biomedical signals (e.g. prediction of pathological stages, ...)
 - biomedical images (e.g. explain Images, video, ...)

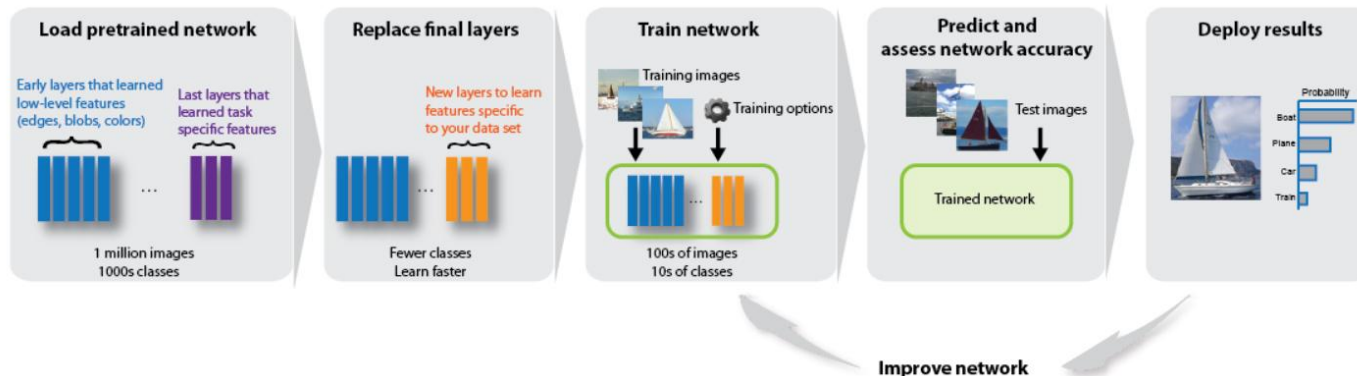
...

Tips and Tricks

Transfer learning

- It turned out, that early layers have a tendency converge to impulse responses corresponding the edge detectors (first and second derivation, edge detectors, Gabbors' or Haars' masks).
- These basic features are then propagated to following layers, and there are combining into complex (abstract) features.
- Is the process of taking a pre-trained model (the weights and parameters of a network) and fine-tuning the model with our own dataset

Reuse Pretrained Network



Pre-trained networks

- most standard publicly available architectures (Alexnet, ResNet, Inception,...)
- Pre-trained on ImageNet dataset
- classification task – to assign into 1000 categories

- We use a same architecture, and we utilize weights of neurons as initial values for fine-tuned network on our dataset

Cons.

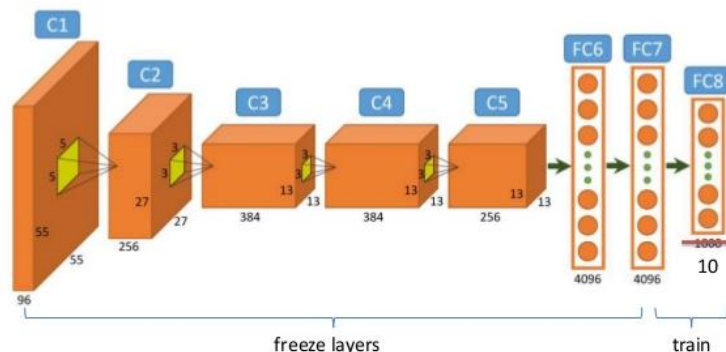
- We must use the same architecture, such as was pre-trained
- The solution to our problem can be other (e.g. more complicated or non standard,...)

Pros.

- The training process requires lower computed time
- Solution more complex problems (with local extreme in cost function)
- Avoids overfitting

Freeze approach

- use a network trained on unrelated categories in a massive dataset (usually Imagenet) and apply it to our own problem because there are universal, low-level features shared between images
- The idea is the convolutional layers extract general, low-level features that are applicable across images — such as edges, patterns, gradients — and the later layers identify specific features within an image such as eyes or wheels.
- We can **freeze** the early convolutional layers of the network and only train the last few layers which make a prediction
- n early layers can be frozen or fine-tuned
- **Frozen** – not updated during backprop.
- **Fine-tuned** – updated during backprop.



ImageNet (and other public datasets)

- is image dataset organized according to the WordNet hierarchy
- there is provided on average 1000 images to illustrate each synset
- Total number of non-empty synsets: 21,841
- Total number of images: 14,197,122
- Number of images with bounding box annotations: 1,034,908

Statistics of high level categories

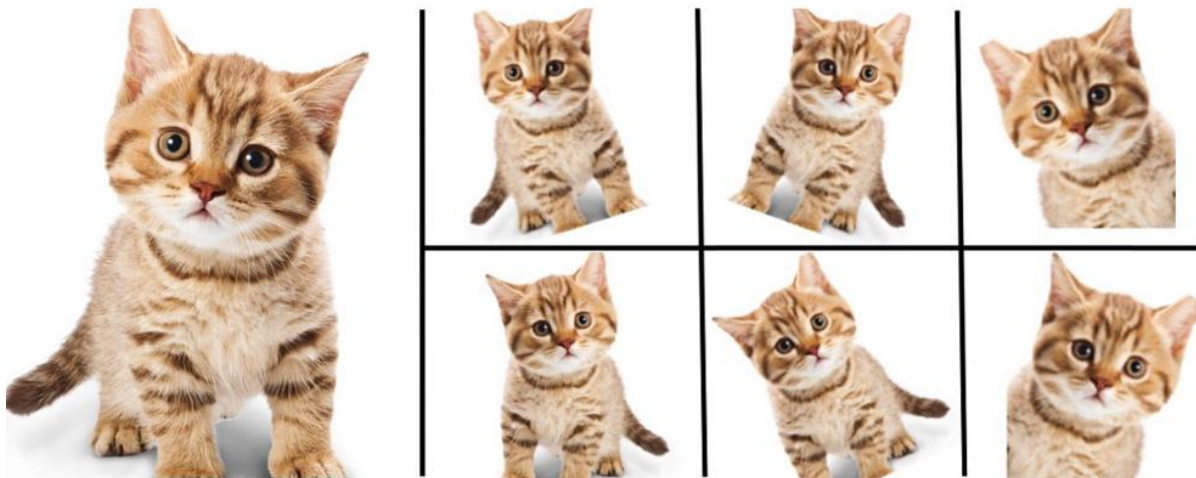
High level category	# synset (subcategories)	Avg # images per synset	Total # images
amphibian	94	591	56K
animal	3822	732	2799K
appliance	51	1164	59K
...	...	...	...

- Most often, the 1000 categories is used for machine learning

Data augmentation

- It is the process for extending the training dataset, which leads to more robust NN.
- We create synthetic variants (synthetically modified) from real images, which can occur in the real world, but it is not available in our dataset.
- **Types:**
 - geometrical transformation (flip, rotation, scaling, crop, ...)
 - intensity/contrast modification (point-wise operation, color,)
 - noising
 - blurring/sharpening
 - data mixing
 - data masking/erasing
 - random image generation (GAN)
- **Realization:**
 - **offline approach** (smaller datasets) before we feed the data to the model.
 - **online approach** is augmentation on the fly, the transformations are performed on the mini-batches (GPU).

Data augmentation



Enlarge your Dataset

Weak- and Self-Supervised Learning

Self-Supervised Learning (SSL)

- Learns from unlabeled data by creating pseudo-labels from the data itself.
- Pretext tasks (e.g., image inpainting, rotation prediction, contrastive learning) guide representation learning.
- Produces generalizable feature embeddings useful for downstream tasks with few labels.
- Reduces the need for costly manual annotation.

Weak-Supervised Learning (WSL)

- Learns from imperfect or limited supervision (noisy labels, incomplete labels, inexact labels (e.g., image-level labels instead of pixel-level))